



PROOFPOINT CYBERSECURITY ACADEMY · CERTIFIED AI AGENT SECURITY SPECIALIST 2026 · SESSION 3

The Agentic Workspace: Runtime Security

Six ways an agent can go wrong once it is live and acting on real systems, and the controls that keep its access, reasoning, credentials and dependencies in check.

PRESENTERS

Patrick Pushor (Director, Solution Architecture) · Tim Choi (GVP, Product Strategy and GTM, AI Security)

CONTENTS

- 01 At a glance
- 02 Over-privileged agents
- 03 Prompt injection & tools
- 04 APIs, credentials & secrets
- 05 Autonomy without oversight
- 06 Model supply chain
- 07 Uncontrolled data sources

QUICK RECALL

Give agents the least access that works, keep a human gate on anything that writes or can't be undone, and log every tool call back to the person and intent behind it.

When the agent is live, the risk is in what it does

Session 2 was about seeing AI use. Session 3 moves to runtime: agents that are already deployed, holding credentials and taking actions. The session walks through six scenarios created by new agentic capabilities, each with its risks, technical controls, and people and process controls.

SCENARIO	WHAT GOES WRONG	FIRST CONTROL TO REACH FOR
Over-privileged agents	Broad access turns one agent into a high-privilege automation layer	Least privilege and separate identities per tool
Prompt injection & tool manipulation	Embedded instructions steer the agent's reasoning and tool calls	Separate reasoning from execution, allowlist tools
Insecure APIs, credentials & secrets	Stolen keys give direct backend access	Vaulted, short-lived, rotated credentials
Autonomy without oversight	Mistakes spread at machine speed with no checkpoint	Human approval gates and a kill switch
Model supply chain	Upstream models and plugins change behavior silently	Pinned, verified dependencies and an SBOM
Uncontrolled data sources	MCP and retrieval links become open pipes to core data	Per-request authorization scoped by data tier

Over-privileged agents

Agents usually run on service accounts or API tokens that reach ticketing, repositories, databases and collaboration tools, and those permissions often copy broad admin or power-user roles just to make sure the agent works. Because an agent acts at machine speed and scale,

loose scoping turns it into a high-privilege automation layer across the company, and the blast radius grows sharply if it is manipulated, compromised or misconfigured.

Risks

Large-scale data exposure across connected systems; unauthorized changes to or deletion of critical records; escalation beyond the agent's intended job; lateral movement through shared service credentials; harmful actions automated at scale; weak accountability when everything is logged under one generic service account.

Technical controls

Design every agent account and token for least privilege, give each tool or system its own identity, use just-in-time access or short-lived tokens for risky operations, enforce fine-grained authorization at the API and tool-call layer, require step-up authentication or approval for destructive actions, and log each action against the user intent and session behind it.

People & process

Review privilege design before deployment, document a business reason for every system and permission, name an owner for each agent identity, recertify agent permissions quarterly, include over-privilege in threat modeling, and put capability expansions under change control.

03 • SCENARIO 2

Prompt injection and tool manipulation

Agents take in external content, user input, retrieved documents and tool responses as part of their reasoning loop, and malicious instructions hidden in any of these can read as legitimate directions. Since agents decide for themselves which tools to call and what data to fetch, the reasoning layer becomes the target: an attacker can steer both decisions and downstream tool calls without exploiting any traditional software flaw.

Risks

Unauthorized retrieval within the agent's permission scope; tool calls that create, change or delete records; cross-system aggregation beyond what the user wanted; guardrails overridden by embedded text; malicious workflows running at machine speed; exploitation that is hard to spot inside legitimate content.

Technical controls

Sanitize prompts and detect injection before reasoning or tool selection, enforce tool-call allowlists with schema and parameter validation, keep reasoning and execution in separate layers so unvalidated model output can't trigger actions directly, apply least privilege to connected systems, gate high-impact or write-capable calls behind human approval, and log reasoning traces, tool choices and retrieval paths.

People & process

Add injection threat modeling to design reviews, teach developers to treat all external content as untrusted, set secure coding standards for tool invocation, run red team exercises against agent workflows, write an AI manipulation response playbook, and assign owners for testing agent defenses.

04 • SCENARIO 3

Insecure APIs, credentials and secrets

Agents lean on API keys, service accounts and OAuth tokens to reach models and systems, and those secrets often end up in config files, environment variables, CI/CD pipelines or orchestration layers. Because one agent automates work across many systems, it tends to hold elevated, cross-platform credentials, which makes it a concentration point for secrets. Steal the credential and you get backend access without touching the model at all.

Risks

Hardcoded keys in source repositories; long-lived credentials that widen the compromise window; tokens reused across environments; direct API access that skips the agent's own logic; secrets leaking through logs, errors and debug output; escalation through broadly permissioned service accounts.

Technical controls

Keep every secret in a central vault with encrypted retrieval, use short-lived tokens that rotate automatically, prefer workload identity federation over static keys, separate dev, test and prod with distinct credentials, restrict where agent credentials can be used through outbound network policy, and scan repositories and pipelines continuously for exposed secrets.

People & process

Write secret management into development standards, peer-review credential handling in code changes, give every service account and token an owner, audit credential hygiene regularly, rehearse secret compromise in tabletops, and include rotation and revocation in incident response plans.

05 • SCENARIO 4

Autonomous action without adequate oversight

Agents are built to reason, decide and act with little human involvement, and to save time companies let them write data, trigger workflows, approve transactions and change system state on their own. Automation is the point, but it also removes the approval checkpoints and human judgment that used to catch errors, and because agents run continuously at machine speed, a mistake or manipulation spreads fast. The job is to keep autonomy from outrunning governance.

Risks

High-impact actions with no second check; errors or malicious changes spreading across systems; financial or contractual commitments made automatically; unapproved configuration changes in infrastructure or SaaS; weak accountability without explicit human confirmation; automated changes that are hard to roll back once widespread.

Technical controls

Require human approval for high-risk or write-capable operations, use risk scoring to decide when step-up authorization applies, set guardrails by action type (read, write, destructive), cap transactions and rates, keep object-level change logs, and build rollback and kill-switch mechanisms that stop an agent immediately.

People & process

Define which actions may run fully autonomously and which need approval, have a governance committee review any expansion of agent authority, map agent actions to business owners, assess risk before enabling new autonomous capabilities, review agent behavior in operational risk audits, and write automated rollback into change management.

06 • SCENARIO 5

Model supply chain and dependency risk

Agents are built on external models, open-source frameworks, orchestration libraries, plugins and third-party tool integrations that change quickly and sometimes update themselves. Unlike ordinary software dependencies, a model can behave very differently between versions with no obvious code change, and agents may load new tools or model providers on the fly. An agent is only as trustworthy as the upstream models and dependencies it runs on.

Risks

Malicious or backdoored agent frameworks and plugins; behavioral drift when a model update changes reasoning or safety limits; hidden flaws in orchestration libraries and tool adapters; compromised third-party model APIs; dependency confusion and typosquatting in package repositories; little visibility into how upstream changes affect agent decisions.

Technical controls

Pin dependency versions and verify integrity with hashes and signed packages, keep an SBOM that covers models, plugins and orchestration layers, stage rollouts with regression testing for upgrades, track provider advisories and disclosures, block dynamic plugin loading unless approved, and monitor agent behavior at runtime after every update.

People & process

Security-review new frameworks and plugins before use, run model and dependency updates through a controlled upgrade process, assign owners for third-party advisories, bring AI components into third-party risk management, reassess model providers regularly, and include supply chain compromise in incident response exercises.

07 • SCENARIO 6

Uncontrolled data source integration

Agents connect to internal systems through MCP servers, retrieval layers, vector databases, SaaS APIs and custom tool adapters, giving them live, queryable access to documents, databases, tickets, repositories and business systems. Unlike a static prompt, these links are persistent pathways between enterprise data and an AI reasoning engine, and without governance they turn an assistant into a high-speed interface to core business data. The question is what the agent can reach, under which conditions, and with how much downstream authority.

Risks

Broad, unscoped retrieval across sensitive repositories; correlation of confidential data that was never combined before; long-lived service accounts on high-value systems; exfiltration through reasoning loops or chained retrieval; exposure of regulated or customer-restricted datasets; write-capable integrations that modify records without authorization.

Technical controls

Scope least privilege per MCP connector and per dataset, segment access by classification tier (public, internal, confidential, restricted), check authorization on every request instead of granting blanket connector access, run content-level DLP before retrieved data enters the model context, keep immutable logs of every object read or changed through MCP, and require human approval for write-capable tool calls.

People & process

Get data owner approval before exposing any system to an agent, keep a registry of active MCP connectors and their scopes, include AI-connected systems in data governance and privacy reviews, recertify access to agent-reachable datasets, threat-model new connectors before production, and assign clear owners for monitoring and revoking unsafe integrations.

The fundamental challenge is controlling what data the agent can access, under what conditions, and with what downstream authority.

SESSION 3 · UNCONTROLLED DATA SOURCE INTEGRATION